armish Processor

Karlo Godfrey Escalona Gregorio

Contents

1 Preface

This project explores a custom implementation of a subset of the ARMv4 instruction set. The instruction set architecture is designed with a custom assembler to make it capable to write programs based on the instruction set. The architecture is implemented in hardware as a custom RTL model, whose functionality is verified.

The assembler is implemented in Python, and the RTL model is implemented using SystemVerilog, using an Arty-S7 25 as a target hardware to use as an example.

This architecture is an educational project inspired by ARM-style RISC design using the ARM7TDMI-S data sheet as a reference. It is not ARM-compatible and does not use proprietary ARM encoding or IP.

2 ISA Design

All instruction words are designed to be 32 bits wide. Each instruction has 4 condition bits that will determine whether or not the instruction executes based on CPSR condition flags (N, Z, C, V). This makes it simpler to write conditional statements for simple instructions. A list of the condition codes is listed below.

Field List			
Condition Code	**Instruction Suffix**	**Flags Set (NZCV)**	**Explanation**
0000	halt	N/A	Terminate program
0001	al	flags ignored	Always Executed
0010	le	Z set OR (N not equal to V)	Less Than or Equal
0011	gt	Z clear AND (N equals V)	Greater Than
0100	lt	N not equal to V	Less Than
0101	ge	N equals V	Greater Or Equal
0110	ls	C clear or Z set	Unsigned Lower or Same
0111	hi	C set and Z clear	Unsigned Higher
1000	vc	V clear	No Overflow
1001	vs	V set	Overflow
1010	pl	N clear	Positive or Zero
1011	mi	N set	Negative
1100	cc	C clear	Unsigned Lower
1101	cs	C set	Unsigned Higher or Equal
1110	neq	Z clear	Not Equal
1111	eq	Z set	Equal

The halt condition code is used to terminate the program. Instructions attached with halt will not execute.

2.1 RX-type: Integer Operations

2.1.1 Overview

The RX-type instructions are used for integer arithmetic data-processing instructions. A summary of the format can be seen in Figure 1, and explanations of the fields can be seen under the figure.

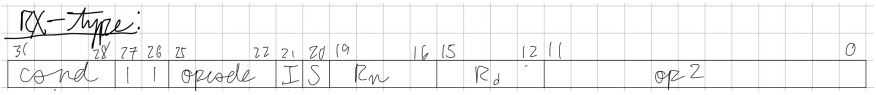


Figure 1: RX-type format for integer instructions.

Field List		
Field	**Bits**	**Description**
cond	[31:28]	State of CPSR condition codes (based on NZCV flags)
type	[27:26]	Encoding specific to instruction type
opcode	[25:22]	Determines the operation performed on operands
I	21	Determines whether or not op2 is an immediate (I = 0 means op2 is not an immediate, but a shift register)
S	20	Determines whether or not to alter condition codes (S = 0 means do not alter)
R_n	[19:16]	First source register
R_d	[15:12]	Destination register
$op2$	[11:0]	Varying field depending on the instruction

RX-type instructions have a varying $op2$ field that can be used depending on whether or not the instruction uses an immediate. For each of the R-type instructions, a closer look will be given in their individual instruction sections.

Field List		
Instruction	**Bits**	**Description**
shift	[11:4]	Used for instructions using two source registers. The amount to shift the value in R_m
R_m	[3:0]	Used for instructions using two source registers. The second source register
rotate	[11:8]	Used for instructions using one source register and one immediate. Rotates the immediate a specific number of positions
imm	[7:0]	A constant used with another shift register to produce the result

Instructions take the following form:

```
(mneumonic)-(instruction suffix) (rd), (rn), (rm)
```

where in each parentheses:

- mneumonic - the type of instruction (e.g. add, sub, etc.)
- instruction suffix - the instruction suffix that details the condition that the instruction is executed under
- rd - destination register

- rn - source register 1
- rm - source register 2/immediate

A list of suported instructions is listed below. It should be noted that because of some complex instructions, the ALU is pipelined to [insert how many stages here] stages.

Instructions		
Mneumonic	**opcode**	**Description**
addx	0000	Adds two integer values
subx	0001	Subtracts two integer values
mulx	0010	Multiplies two integer values
divx	0011	Divides two integer values
absx	0100	Takes the absolute value of an operand
adcx	0101	Adds two integer values and a carry flag from a previous instruction
sbcx	0110	Subtracts two integer values and a carry flag from a previous instruction
cmpx	0111	Compares two operands and outputs the appropriate flag
notx	1000	Takes the bitwise NOR of two operands
andx	1001	Takes the bitwise AND of two operands
orrx	1010	Takes the bitwise OR of two operands
xorx	1011	Takes the bitwise XOR of two operands

2.1.2 addx/subx

The addx and subx instructions add or subtract two numbers and store them into a destination register. The following snippet shows the cases for add, but sub follows a similar format.

```
// add the values stored in r1 and r2 and store them
   into r3
addx.s-al r3, r1, r2
// add 8 to the value stored in r1 and store them into r3
addx.s-al r3, r1, #8
// add the values stored in r1 and r2 and store them
   into r3, and use the result to set NCZV flags
addx.s-al r3, r1, r2
// r3 = r1 - r2
subx-al r3, r1, r2
```

The $op2$ field in the instruction format for add/sub takes on different forms depending on the value of of bit 25 (I). For I=0, the $op2$ field operates under the assumption that the 3rd operand is stored in a register. For I=1, the $op2$ field operates under the assumption that the 3rd operand is an immediate value.

3rd Operand: Register

When the 3rd operand is a register, the value in the register can be manipulated through shifting before carrying out addition or subtraction.

```
// add the values stored in r1 and r2 (whose value is
   shifted logically to the left by a value specified in
   r4) and store them into r3
addx r3, r1, r2, lsl r4
// add the values stored in r1 and r2 (whose value is
   shifted logically to the left by 8) and store them
   into r3
addx r3, r1, r2, lsl #8
```

The $op2$ field specifications are as follows:

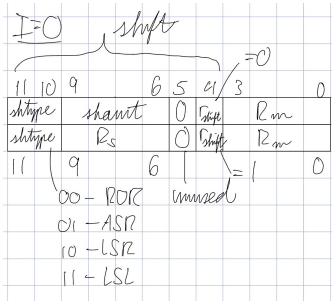


Figure 2: $op2$ field when the 3rd operand is a register. The top field is the format when the 3rd operand is shifted by a constant. The bottom field is the format when the 3rd operand is shifted by an amount specified in a register.

Field List for $op2$ (3rd operand register, shifted by immediate)		
Field	**Bits**	**Description**
shtype	[11:10]	The shift type performed on the 3rd operand
shamt	[9:6]	The amount that the 3rd operand is shifted by
unused	5	unused
r_{shift}	4	The bit that specifies whether the shifting operand is a register or an immediate (value after lsl)
R_m	[3:0]	The register holding the second operand

Field List for $op2$ (3rd operand register, shifted by register value)		
Field	**Bits**	**Description**
shtype	[11:10]	The shift type performed on the 3rd operand
R_s	[9:6]	The register that contains the amount that the 3rd operand is shifted by
unused	5	unused
r_{shift}	4	The bit that specifies whether the shifting operand is a register or an immediate (value after lsl)
R_m	[3:0]	The register holding the second operand

The shift type (shtype) determines what kind of shift the second operand goes through. The specifications for the shift type are as follows:

Description of Shift Types		
Shift Type	**Encode**	**Description**
ror	00	Rotate right
asr	01	Arithmetic shift right
lsr	10	Logical shift right
lsl	11	Logical shift left

For carrying out the operation without any shifting, it is sufficient to just not include a mention of the shift. It will assume lsl #0, which will not perform any shift.

3rd Operand: Immediate When the 3rd operand is an immediate, the values the immediate can take a variety of values.

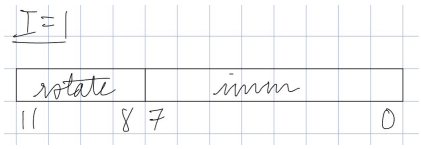


Figure 3: $op2$ format for when the 3rd operand is an immediate.

Field List for $op2$ (3rd operand immediate)		
Field	**Bits**	**Description**
rotate	[11:8]	Number defining how many times the immediate is rotated right in a 16-bit value
imm	[7:0]	Immediate to be encoded

The 8-bit immediate field can be used to values from 0 to 255. Since each register is 16-bits, an 8-bit immediate isn't enough to reach the full value range that can be held by the register. With the rotate field, the rest of the bits in each register can be set, and a wider range of immediates can be used.

Figure 4: How the value of rotation affects which bits are selected to be affected

One unique difference from ARMv7 is that the rotation encoding from an 8-bit immediate into 16-bits is that this allows for a wider range of access of immediates possible for 16-bit operands, meaning the effective range of encoding immediates at the cost of higher hardware complexity. Because the registers store signed values, the effective range is 0 to $2^{15} - 1$.

It should be noted that negative immediates are not directly supported, but negative values can be indirectly made through subx.

2.1.3 adcx/sbcx

The adcx and sbcx instructions add or subtract two numbers with a carry from the previous instruction and store them into a destination register. The formatting used for the instruction is the same as addx/subx, with the only distinction between the instruction being the opcode.

```
// add the values stored in r1 and r2 and store them
   into r3
adcx.s-al r3, r1, r2
```

```
// add 8 to the value stored in r1 and store them into r3
adcx.s-al r3, r1, #8
// add the values stored in r1 and r2 and store them
   into r3, and use the result to set NCZV flags
adcx.s-al r3, r1, r2
// r3 = r1 - r2
sbcx-al r3, r1, r2
```

2.1.4 mulx/divx

The mulx instruction can multiply two numbers and store them into a destination register.

```
// multiply the values stored in r1 and r2 and store the
   product into r3 and r4
mulx-al r3, r4, r1, r2
// divide the values stored in r1 and r2 and store the
   quotient into r3 and the remainder in r4
divx-al r3, r4, r1, r2
// integer division of r1 and r2 and store the quotient
   into r3
divx-al r3, r1, r2
```

The $op2$ format for mulx is shown below. For full context, part of the rest of the instruction encoding is also shown.

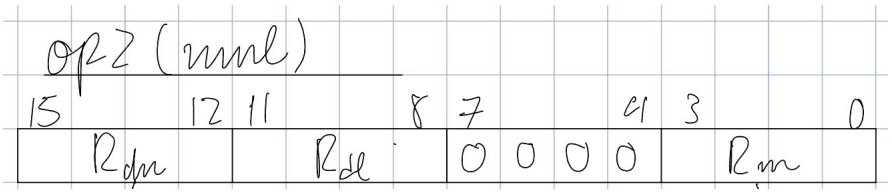


Figure 5: $op2$ encoding for the mul instruction

Field List for $op2$ (mul)		
Field	**Bits**	**Description**
R_{du}	[15:12]	Register to hold the upper byte of the product (technically not part of $op2$
R_{dl}	[11:8]	Register to hold the lower byte of the product
unused	[7:4]	unused
R_m	[3:0]	The register holding the second operand

Because the product of 2 16-bit numbers is 32-bit, two registers are necessary to hold the entire product.

The $op2$ format for div is shown below. For full context, part of the rest of the instruction encoding is also shown.

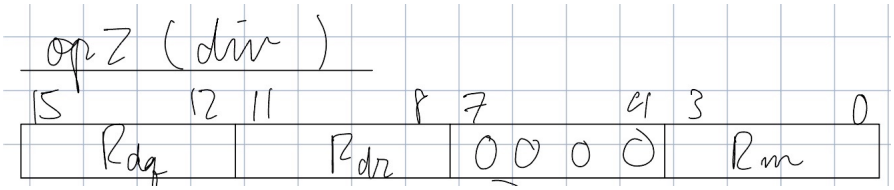


Figure 6: $op2$ encoding for the div/divi instructions.

Field List for $op2$ (div)		
Field	**Bits**	**Description**
R_{dq}	[15:12]	Register to hold the quotient (technically not part of $op2$)
R_{dr}	[11:8]	Register to hold the remainder
rem	7	Bit to decide whether or not to keep the remainder (rem = 1 means keep the remainder)
unused	[6:4]	unused
R_m	[3:0]	The register holding the second operand

One register is used to store the quotient, and 1 register is used to store the remainder of the division.

Some things to note about mul/div:

- Immediates cannot be used. The 32-bit instructions doesn't have the capacity to use immediates.
- NCZV flags cannot be set with mul and div. Allowing for this increases the complexity of the hardware by too much. This means S is always set to 0
- Trying to divide by 0 will set both the product and remainder to 32'hFFFF˙FFFF

2.1.5 absx

The absx instruction takes the absolute value of a register.

```
// take the bitwise not of r1 and store into r2
notx-al r2, r1
```

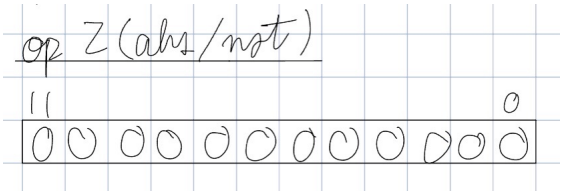


Figure 7: $op2$ encoding for the not instruction

$op2$ is set to all 0s, since not is a unary operator. This also means that immediates have no purpose for this instruction, as well as setting NCZV flags (I = 0, S = 0).

2.1.6 cmpx

The cmpx instruction compares to registers.

```
// compares r2 and r1 and sets the appropriate flags
cmpx-al r2, r1
// compares r2 and 145 and sets the appropriate flags
cmpx-al r2, #145
```

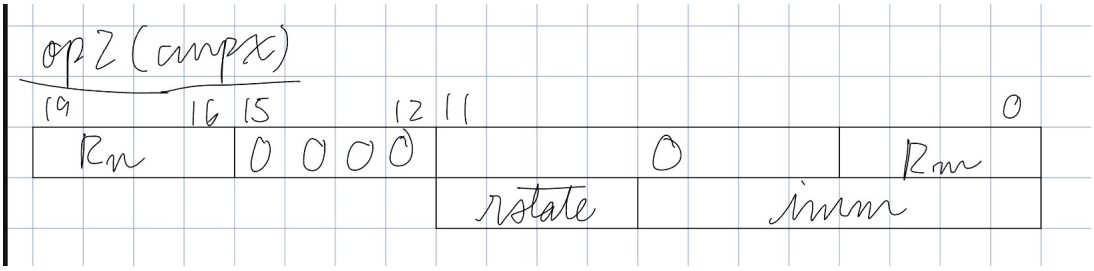


Figure 8: Encoding for bits [19:0] for cmpx

For $op2$, cmpx uses the same immediate encoding as add/sub instructions (see 2.1.2), allowing for both register and immediate comparison. R_d is set to 0.

Field List for $op2$ of cmpx (3rd operand register I = 0)		
Field	**Bits**	**Description**
r0	[15:12]	The r0 register to send the final result to (to be ignored)
0	[11:4]	Zero Filling
imm	[3:0]	Immediate to be encoded

Field List for $op2$ of cmpx (3rd operand immediate I = 1)		
Field	**Bits**	**Description**
r0	[15:12]	The r0 register to send the final result to (to be ignored)
rotate	[11:8]	Number defining how many times the immediate is rotated right in a 16-bit value
imm	[7:0]	Immediate to be encoded

The destination register is set to the 0 register, which is always hardwired to 0 value.

Some notes about cmpx:

- cmpx will always set flags. Do not add '.s'

2.1.7 notx

The notx instruction can take the bitwise not of what is stored in the source register.

```
// take the bitwise not of r1 and store into r2
notx-al r2, r1
```

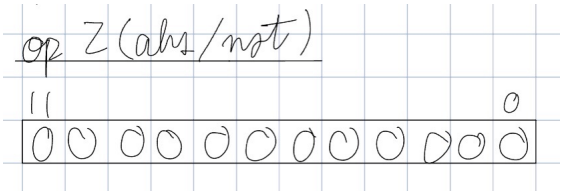


Figure 9: $op2$ encoding for the not instruction

$op2$ is set to all 0s, since not is a unary operator. This also means that immediates have no purpose for this instruction, as well as setting NCZV flags (I = 0, S = 0).

2.1.8 andx/orrx/xorx

The andx instruction can take the bitwise and of what is stored in the source register. The orrx instruction can take the bitwise or of what is stored in the source register.

```
// take the bitwise and of r1 and r2 and store into r3
andx-al r3, r1, r2
// take the bitwise and of r1 and #0x00ff and store into
   r3
andx-al r2, r1, #9
```

```
// take the bitwise or of r1 and r2 and store into r3
orrx-al r3, r1, r2
// take the bitwise or of r1 and #0x00ff and store into
   r3
orrx-al r2, r1, #0x00ff
// take the bitwise xor of r1 and r2 and store into r3
xorx-al r3, r1, r2
// take the bitwise xor of r1 and #0x00ff and store into
   r3
xorx-al r2, r1, #0x00ff
```

The encoding done for $op2$ is identical to that of addx/subx instructions, so shifting operations can be applied to the 3rd operand (given that it is a register), if desired (see 2.1.2).

2.1.9 Miscellaneous Notes: R-type Fixed-Point Instructions

A few things to note about R-type instructions:

- To update NCZV flags after addx, subx, append an 's' after the mneumonic (i.e. adds, subs).
- For operations that take a shifting argument, it makes no sense to shift more than 15 positions. Therefore, shift inputs more than 15 will force the register value to be 0.
- Even though some RX instructions don't use immediates (mulx, divx), I will always be set to 1 to simplify shifting hardware.

2.2 RF-type: Floating Point Operations

Note: This section will be implemented if time allows for it. Implemented as a coprocessor according to ARM7-TDMI-S (Ch 4

2.2.1 Overview

The RF-type instructions are used for floating-point arithmetic data-processing instructions, using the IEEE-754 floating-point standard format. A summary of the format can be seen in Figure 3, and explanations of the fields can be seen under the figure.

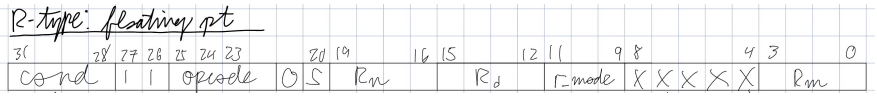


Figure 10: RF instruction type format.

Field List		
Field	**Bits**	**Description**
cond	[31:28]	State of CPSR condition codes (based on NZCV flags)
type	[27:26]	Encoding specific to instruction type
opcode	[25:22]	Determines the operation performed on operands
unused	21	unused
S	20	Determines whether or not to alter condition codes (S = 0 means do not alter)
R_n	[19:16]	First source register
R_d	[15:12]	Destination register
r_{mode}	[11:9]	Specifies the rounding mode of the floating point operation. See the underlying table for details.
unused	[8:4]	unused (might do flags for invalid operations)
R_m	[3:0]	Varying field depending on the value of opcode

r_{mode}	
r_{mode} **value**	**Description**
000	Operation rounds toward 0
001	Operation rounds toward nearest, ties away from 0
010	Operation rounds toward nearest, ties to even
011	Operation rounds toward $+\infty$
100	Operation rounds toward $-\infty$

Instructions take the following form:

```
(mneumonic).f-(instruction suffix) (rd), (rn), (rm),
   #(r_mode)
```

where in each parentheses:

- mneumonic - the type of instruction (e.g. add, sub, etc.)
- instruction suffix - the instruction suffix that details the condition that the instruction is executed under
- rd - destination register
- rn - source register 1
- rm - source register 2
- r_{mode} - the rounding mode for the floating point operation

Instructions take the following form:

```
(mneumonic).f-(instruction suffix) (rd), (rn), (rm),
   #(r_mode)
```

where in each parentheses:

- mneumonic - the type of instruction (e.g. add, sub, etc.)
- instruction suffix - the instruction suffix that details the condition that the instruction is executed under
- rd - destination register
- rn - source register 1
- rm - source register 2
- r_{mode} - the rounding mode for the floating point operation

A list of suported instructions is listed below.

Instructions		
Field	**opcode**	**Description**
addf	1000	Adds two floating-point values
subf	1001	Subtracts two floating-point values
mulf	1010	Multiplies two floating-point values
divf	1011	Divides two floating-point values
cmpf	1100	Compares two floating-point values)
cnvf	1101	Convert value to IEEE-754 floating-point standard format
sqrf	1110	Takes square root of a floating-point value
recf	1111	Takes reciprocal of a floating-point value

A few things to note about RF-type instructions:

- The instructions cannot be used to set CPSR condition codes, and are undefined for immediate type instructions.
- To choose the rounding mode for the floating point operations, after the '.f' market, use '#' followed by the value of r_{mode} to specify the rounding operation (e.g. add.f-al r1, r2, r3, #4 to round toward $-\infty$).
- Rounding mode is underfined for cmp instruction. Just only use the two operands being compared
- Note the lack of immediate operations. To use immediate values, use fixed-point representation to create the immediate value with addi, and then convx2f.

2.3 D-type: Data Movement Operations

2.3.1 Overview

The D-type instructions are used for loading and storing data from and into memory.

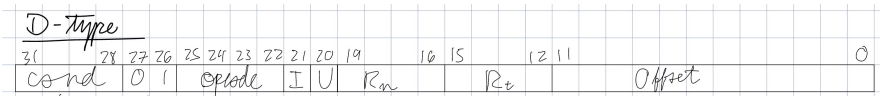


Figure 11: D instruction type format.

Field List		
Field	**Bits**	**Description**
cond	[31:28]	State of CPSR condition codes (based on NZCV flags)
type	[27:26]	Encoding specific to instruction type
opcode	[25:22]	Encoding specific to instruction
I	21	Determines whether or not the the offset is an immediate value or a register (I = 1 means that it is an immediate value, I = 0 means that the offset is stored in a register)
U	20	Determines whether the offset is added or subtracted (U = 1 means that the offset is added, U = 0 means that the offset is subtracted)
R_n	[19:16]	Address register used to interact with memory
R_t	[15:12]	Transfer register used to read/write with memory
offset	[11:0]	Offset used to calculate where to load/store data. For a register offset, the register would be the least significant 4 bits

Instructions take the following form:

```
(mneumonic)-(instruction suffix) (rd), [(rn), (offset)]
```

where in each parentheses:

- mneumonic - the type of instruction (e.g. add, sub, etc.)
- instruction suffix - the instruction suffix that details the condition that the instruction is executed under
- rd - Register in register file to load or store to
- rn - Register holding the address to interact with in data memory

- offset - offset used to calculate where to load/store data

The opcode has an encoding that is like so:

opcode Field List		
Field	**Bits**	**Description**
Load/Store	25	Tells whether it is a load or a store
Byte/Word	24	Tells whether to load/store a byte or a word
Upper Byte/Lower Byte	23	Tells whether to load/store to/from the upper byte or the lower byte of the register file
0 (Default)	22	Set to 0 by default (unused)

A list of suported instructions is listed below.

Instructions		
Instruction	**opcode**	**Description**
ldw	0110	Loads a 16-bit word from data memory into a register in the register file
ldb2l	0000	Loads a byte from data memory into the lower byte of a register in the register file
ldb2h	0010	Loads a byte from data memory into the upper byte of a register in the register file
stw	1110	Stores a 16-bit word from a register in the register file into data memory
stb2l	1000	Stores a byte from the lower byte of a register in the register file into data memory
stb2h	1010	Stores a byte from the lower byte of a register in the register file into data memory

2.3.2 ld

Load instructions are used to load data from data memory into the register file. Users have the option of loading an entire 16-bit word or just a byte, which can be written into the upper or lower byte of a register. Additionally, it is possible to choose whether or not the offset is defined by an immediate or by a register.

```
// load a 16-bit word from data memory into r2, 2 bytes
   upstream from the address stored in r1
ldw r2, [r1, #2]
```

```
// Load a 16-bit word from memory into r2, 2 bytes
   downstream from the address stored in r1
ldw r2, [r1, #-2]
// Load a 16-bit word from memory into r2, according to
   the value stored in r3
ldw r2, [r1, r3]
// Load a 16-bit word from memory, offset according to
   the value stored in a 4 times shifted version r3
ldw r2, [r1, r3, lsl #4]
// Load the byte stored at address r1 into the lower
   byte of the register r2
ldb2l r2, [r1, #0]
// Load the byte stored at address r1 into the lower
   byte of the register r2
```

Calling ldw will load the byte that is at the given address, and the next byte downstream. This means that if ldw was called on address 0x0000, it will load the byte at that address, as well as the byte at address 0x0001. Calling ldb2l or ldb2h will completely overwrite the register. The byte of data obtained from memory will be zero-extended and be used to overwrite whatever is in the register. The value of the offset can be determined by a register or by an immediate value. These follow the same format as the 3rd operand in R-type instructions like add and sub (See Section 2.1.2).

2.3.3 st

Store instructions are used to store data from the register file into data memory. Users have the option of storing an entire 16-bit word or just a byte, which can be read from the upper or lower byte of a register. Additionally, it is possible to choose whether or not the offset is defined by an immediate or by a register.

```
// store a 16-bit word from r2 into data memory,
   specified by the address given by r1 2 bytes upstream
stw r2, [r1, #2]
// Load a 16-bit word from memory, 2 bytes downstream
stw r2, [r1, #-2]
// Load a 16-bit word from memory, offset according to
   the value stored in r3
stw r2, [r1, r3]
// Load the byte stored at address r1 into the lower
   byte of the register r2
stb2l r2, [r1, #0]
// Load the byte stored at address r1 into the upper
```

```
    byte of the register r2
```

Calling stw will store the register's upper byte in the location of the requested address, and the register's lower byte the next byte downstream. The value of the offset can be determined by a register or by an immediate value. These follow the same format as the 3rd operand in R-type instructions like add and sub (See Section 2.1.2).

2.3.4 Miscellaneous Notes about D-type Instructions

- To specify loading a byte, add a 'b' after the mneumonic (ldrb, strb), otherwise it will default to loading/storing a word.
- To specify whether an offset is added or subtracted, use positive offset values for adding, and negative offset values for subtracting (e.g. ldr r0, [r1, #8] for the address r1 + 8, ldr r0, [r1, #-8] for the address r1 - 8).
- To specify whether an offset is an immediate value or a register, use '#' to specify the offset, or 'r' to specify a register (e.g. ldr r0, [r1, #8] for an offset or ldr r0, [r1, r2] for a register).
- The hardware uses big-endian formatting.
- Because the data memory is 256 words, the highest address is 255. Calling ldw or stw on this address is not supported, and can lead to unpredictable behavior.

2.4 B-type: Branching Operations

2.4.1 Overview

B-type instructions are used for procedure calls. The ISA uses relative branching.

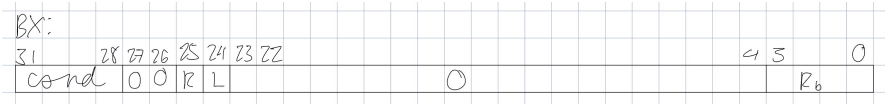


Figure 12: B instruction type format for BX instruction

Field List (BX)		
Field	**Bits**	**Description**
cond	[31:28]	State of CPSR condition codes (based on NZCV flags)
type	[27:26]	Encoding specific to instruction type
R	25	Determines whether the instruction is a BX instruction vs B or BL instructions (R = 0 means that it is a BX instruction, while R = 1 means that it is either a B or a BL instruction)
L	24	Determines whether the instruction is a B or an BL instruction (L = 1 means that it is a BL instruction, and L = 0 means that it is a B instruction)
R_b	[3:0]	Address of the register containing the address to branch to

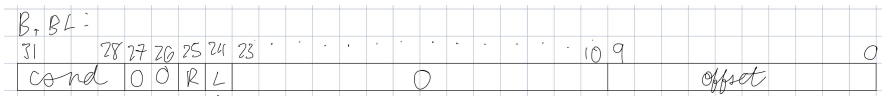


Figure 13: B instruction type format for B and BL instruction

Field List (B or BL)		
Field	**Bits**	**Description**
cond	[31:28]	State of CPSR condition codes (based on NZCV flags)
type	[27:26]	Encoding specific to instruction type
R	25	Determines whether the instruction is a BX instruction vs B or BL instructions (R = 0 means that it is a BX instruction, while R = 1 means that it is either a B or a BL instruction)
L	24	Determines whether the instruction is a B instruction vs a BL instruction (L = 0 means that it is a B instruction, while L = 1 means that it is a BL instruction)
offset	[9:0]	Relative address of the label to branch to

Instructions take the following form:

```
(mneumonic)-(instruction suffix) (label)
```

where in each parentheses:

- mneumonic - the type of instruction (e.g. add, sub, etc.)
- instruction suffix - the instruction suffix that details the condition that the instruction is executed under

- label - the label or register containing program counter value to branch to

Instructions	
Field	**Description**
bx	Branches to an address specified by a register
b	Branch to a label
bl	Branch and link

2.4.2 bx

Branch and exchange is a branching instruction that branches to an address stored in a register. It is commonly used to return from a procedure using the link register (r14).

```
// Return from procedure
bx lr
```

Some things to note:

- This is an absolute branching instruction that branches to the address stored in the register. This means that it just loads the address into the program counter directly.

2.4.3 b

The general relative branch instruction branches to an address stored in a label. For conditional branching, NCZV flags must be set by a previous instruction.

```
// go to label
b label
// go to label if registers r1 and r2 are equal
subs r0, r1, r2
beq label
```

2.4.4 bl

The branch and link instruction stores the address of the next instruction before branching to a label.

```
// go to label and save the location of the instruction
   after the label
bl label
```

2.4.5 Important Notes for B-type Instructions

- B and BL instructions contain a signed 2's complement 10-bit offset.
- A 10-bit offset is used because the intention is for the instruction memory to only have 1024 locations.

2.5 Important Notes

- Labels must be alone on its own line. In other words, this is allowed:

```
label:
addx-al r1, r2, r3
```

  But this is not:

```
label: addx-al r1, r2, r3
```

- Labels don't have a specific syntax defined. As long as the label is before a ':', it is a valid label. Using multiple colons for a label will cause some undefined behavior.
- Negative shift amounts are undefined for instructions that involve shifting
- **All RX-type instructions operate on 16-bit fixed point 2's complement numbers. Likewise, all RF instructions operate on 16-bit floating point numbers. Using an instruction on an incompatible number will yield unexpected results.**
- The only registers that should have unisgned integers should be PC and LR.

3 Assembler

The assembler is implemented as a two-pass assembler in Python. In the first pass, labels are assigned location counter (LC) values to represent where they will be stored in instruction memory. For an instruction memory of 2^{16} addresses, 16 bits are used to represent the addresses. These values are stored in a symbol table implemented as a hash table. In the second pass, all instructions are put into their machine code counterpart in the following format (similar to .bin files):

```
0x##: ## ## ## ##
```

The number before the colon is a hexadecimal representation of the LC value, and the numbers after it are the hexadecimal representation of the instruction encoding. A binary version of this is also produced. Consider the following example instruction:

```
  addx-al r0, #9
```

A few things to note about the assembler:

- Multiple labels of the same have undefined behavior. Since the symbol table was implemented as a Python dictionary, the most recent definition of the label will probably be what defines the label.
- There is nothing to check for invalid syntax. The programmer takes responsibility for making sure everything is correct.
- No bounds checking exists in the assembler.

4 RTL Design - Two-Cycle

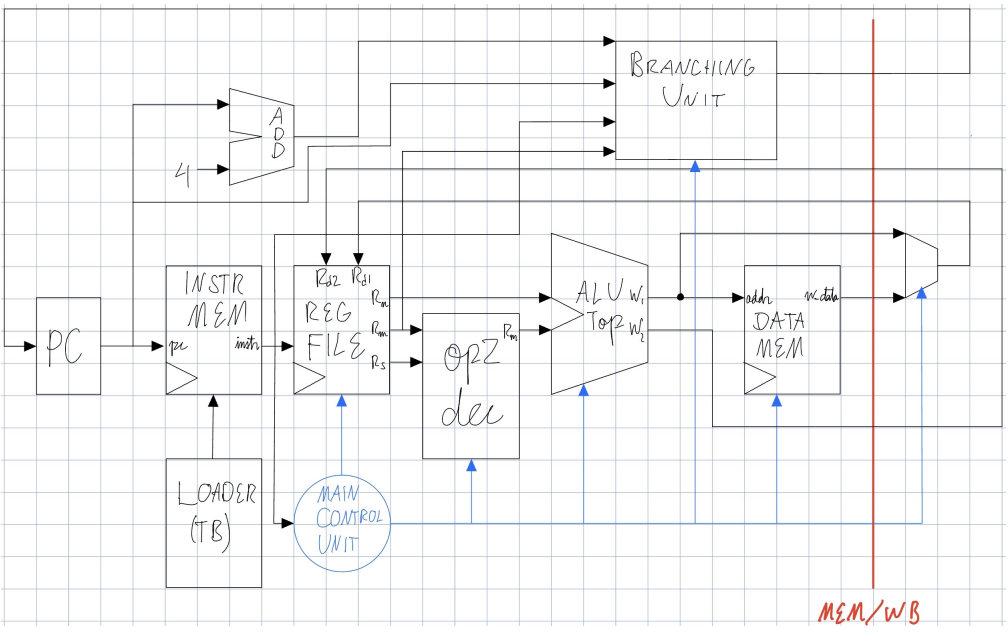
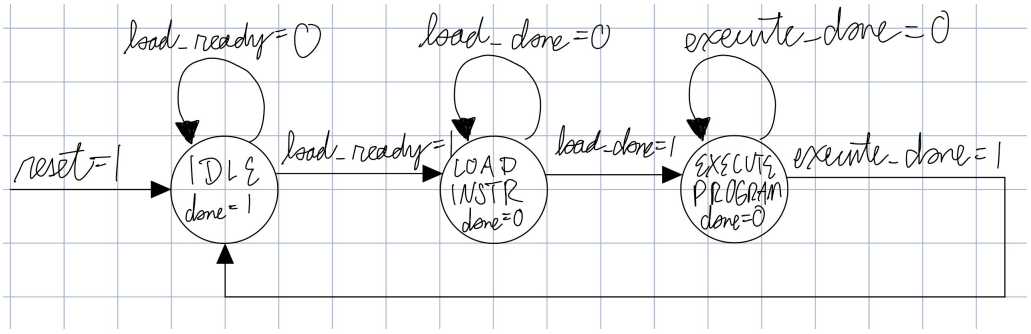


Figure 14: Main datapath and control for the ARMish Processor

This section covers the implementation of the datapath and control for a two-cycle implementation of the processor.

4.1 Program Execution Control

The hardware needs to know when to load the program into instruction memory and when to execute the program. To do this, a simple Moore FSM was used to design the top-level control flow on when to load the instructions and when to execute the program.



4.2 Program Counter Adder

The program counter adder calculates the next value of the program counter, a register that keeps track of the address of the next instruction.

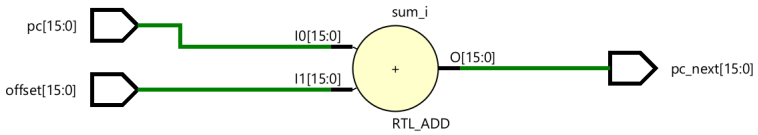
4.2.1 Design

Design Specification

- Purpose and Scope
  - The program counter adder is an adder that calculates the next value of the program counter, a register that keeps track of the address of the current instruction.
  - The adder should produce a new address that will be a possible value for the program counter.
- Functional Requirements
  - Add two 16-bit 2's complement numbers and outputs a positive 2's complement number representing the next value of PC.
  - While the inputs are two 2's complement numbers, the output should always be positive, as addresses can never be negative.
- Interface Specification
  - Inputs
    - PC : 16-bit 2's complement number that represents the current address of the program
    - offset: 16-bit 2's complement number that represents how much to add to PC
  - Outputs
    - PC_next: 16-bit 2's complement number that represents the next address of the program

Implementation

The PC adder is a simpler adder.



4.2.2 Verification

Test Plan

Program Counter Adder Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	Addition	Perform addition of two 2's complement numbers for the following combinations: - Positive number (address) with positive number (offset) - Positive number (address) with negative number (negative number should not be larger than the positive number) (offset)

Tests

The following tests were performed on the adder, and were successfully completed:

- Addition of a positive offset (ranging from 0 to 512) onto a 0 address.
- Addition of a positive offset (ranging from 0 to 1024) onto a positive address (512 in this case)
- Addition of a negative offset (ranging from 0 to -512) on a positive address (512 in this case)

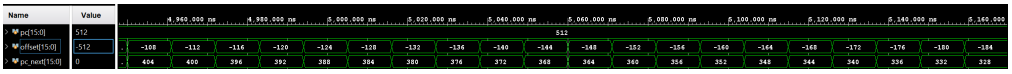


Figure 15: Example waveform output for the pc adder

4.3 Instruction Memory

The instruction memory is a memory that holds the program instructions.

4.3.1 Design

Design Specification

- Purpose and Scope
  - The instruction memory is a memory that holds the program instructions.
  - A central location to store and output instructions is necessary for an organized execution of instructions.
- Functional Requirements
  - Instruction memory is composed of 1024 possible memory locations, each location holding 32-bit instructions.
  - When a write signal is asserted, the instruction memory should be loading instructions from a testbench.
  - When a write signal is deasserted, the instruction memory should be outputting instructions from an address given to it.
  - To be synthesizable, the instruction memory must be able to load instructions from an outside source (test bench)
- Interface Specification
  - Inputs
    - r_address: the address of instruction memory to read from; determines what instruction is outputted (read mode)
    - w_instruction: the instruction to be written into instruction memory (write mode)
    - w_address: the address to be written into instruction memory (write mode)
    - w_e: signal determining whether the instruction memory is in read mode or write mode
  - Outputs
    - instruction: the instruction to be outputted from instruction memory

Implementation

The instruction memory has a memory size of 1024 32-bit words. The instruction memory loads instructions serially. When a w_e is asserted, it will store an inputted instruction at a given address every clock cycle until all words are loaded (load completion is determined by the testbench). After loading, instructions can be accessed by inputting a read address into the instruction memory. Because the instructions are byte addressable, the instruction memory also takes this into account when outputting the instruction.

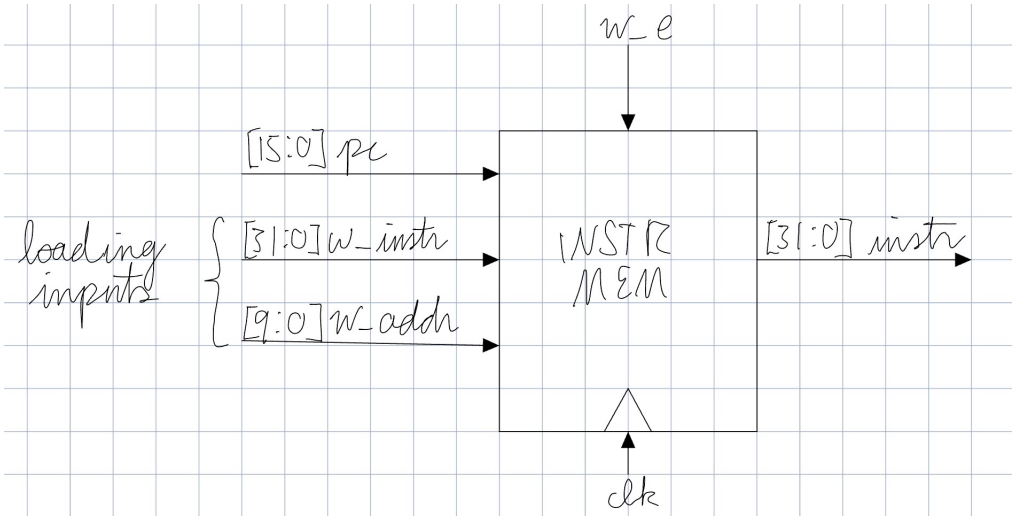


Figure 16: Rough sketch of instruction memory.

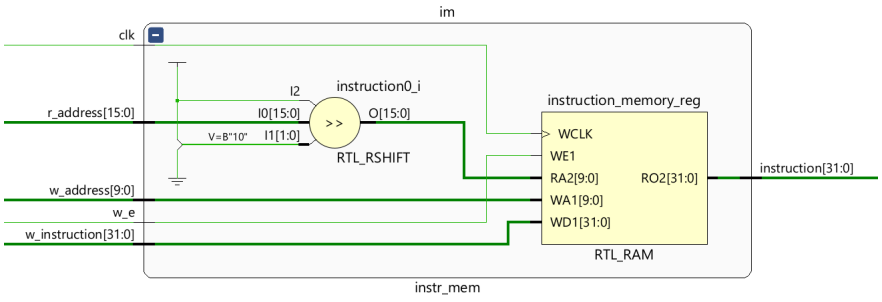


Figure 17: RTL model of instruction mem.

4.3.2 Verification

Test Plan

Immediate Decoder Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	Accurate Immediate Decoding	Valid immediates should be able to be accurately decoded for the entire range specified

Tests

To test these features, it is necessary to have a way for the testbench to communicate with the instruction memory when it is finished writing to it. To do this, 3 tasks were written. The first task reads memory from the machine code file into a memory located in the testbench. The second task counts the number of lines in the machine code file, so the processor knows when loading is completed. The third task loads an instruction into instruction memory every clock cycle.
To test the functionality of the instruction memory, a program of 6 instructions was loaded into the instruction memory, and read in different ways.

```
00011100000100000001111001110001
00011100000100000010000001100011
00011100000100000011000001011001
00011100000100010100000000000010
00011100000101000100111000011011
00011100000101000101000000000010
```

All 6 instructions were loaded into the instruction memory, and then were read sequentially, instructions at even addresses, and then instructions ad odd addresses.

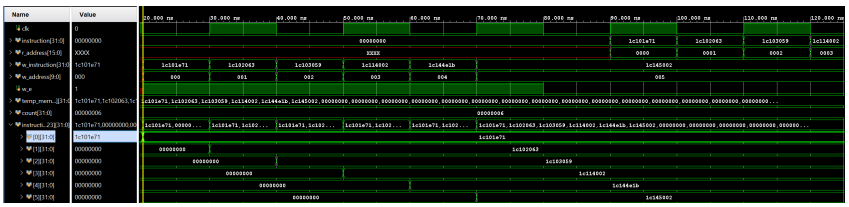


Figure 18: Waveform view of the instruction memory being loaded by instructions specified by w_instruction at address w_address.

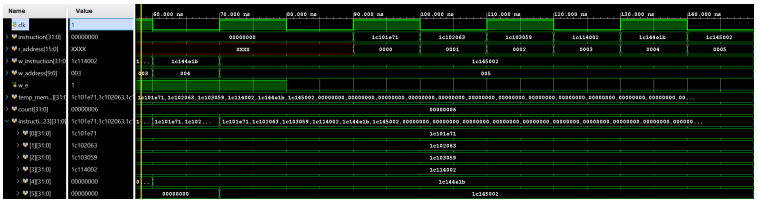


Figure 19: Instructions being read out at addresses specified by r_address.

4.4 Main Register File

A register file is a set of registers that can be quickly accessed to store data.

4.4.1 Design

There are a total of 16 16-bit registers in the main register file, including link register, program counter, and zero/discard register. 16-bit registers were chosen, due to the goal of designing a processor that performs floating point operations, which are too complex to be done in 1 clock cycle for 32-bit operands. 16-bit operands can get very close to IEE-754 compliance. The remaining 12 registers are general-purpose.

Main Register File	
Register	**Purpose**
r0	Zero Register/Discard Register
r1-r13	General Purpose
r14	Link Register
r15	Program Counter (PC)

Design Specification

- Purpose and Scope
  - A central place to store intermediate values is needed to ensure timely calculations.
  - The main register file mainly stores data from fixed point operations. Floating point operations are handled in a separate register file.
  - Same cycle read/write is not covered for now, but will be covered in the pipelining section.
  - It is assumed that an instruction will never use both of its write ports to write to the same register.
- Functional Requirements
  - The register file outputs values when requested (read mode).
    - It should be able to read from 3 registers concurrently to satisfy the needs of the instruction set.
  - The register file stores values when requested (write mode).
    - The register file should be able to overwrite registers.

        * It should be able to write to 2 registers concurrently to satisfy the needs of the instruction set.
    - The register file should be capable of storing 16 16-bit values for all instructions specified by the ISA.
        * For ALU outputs that require more than 16 bits, the outputs should be stored in multiple registers in a known order.
        * r0 should always hold value 0, even when written to.

- Interface Specification
    - Inputs
        * r_reg1: Register to read specified by the 2nd operand of the instruction (R_n)
        * r_reg2: Register to read specified by the 3rd operand of the instruction (R_m)
        * r_reg3: Register to read specified by the 4th operand of the instruction (R_s, often optional)
        * r_reg4: Register to read specified by the 1st operand of the instruction (R_t, only used for store instructions)
        * w_data1: Data to write to register specified by 1st operand of the instruction
        * w_data2: Data to write to register specified by w_reg2 (often optional)
        * w_data3: New LR value to write
        * w_data4: New PC value to write
        * w_reg1: Register to write to specified by 1st operand of the instruction (R_{d1})
        * w_reg2: Register to write to specified by 4th operand of the instruction (often optional) (R_{d2})
        * reg_write1: Signal to allow for writing to the register file through the first write port
        * reg_write2: Signal to allow for writing to the register file through the second write port
        * l: Signal to update link register
        * reset: Synchronous reset, sets all registers to 16'b0
        * clk: clock
    - Outputs

* r_data1: Data stored in register specified by the 2nd operand of the instruction (R_n)
* r_data2: Data stored in register specified by the 3rd operand of the instruction (R_m)
* r_data3: Data stored in register specified by the 4th operand of the instruction (R_s)
* r_data4: Data stored in register specified by the 1st operand of the instruction (R_t)

Implementation

The register file is modeled as a memory with depth 16, each register with width 16. At the positive edge of each clock cycle, it checks for a reset signal. If the reset signal is high, then all the registers in the register file are set to 0. In absence of a reset signal, the register file checks its two reg_write signals to decide whether it needs to write data to the register file. If either signal is high, data will be written to the register corresponding to the reg_write signal that is high. If two writes are done at the same time, and if they are writing to the same register, the data specified by w_reg1 will be prioritized. Data is read by requesting data for the registers specified by r_reg1 and r_reg2. The read is asynchronous, in that it will update combinationally if a write is performed on a register being read.

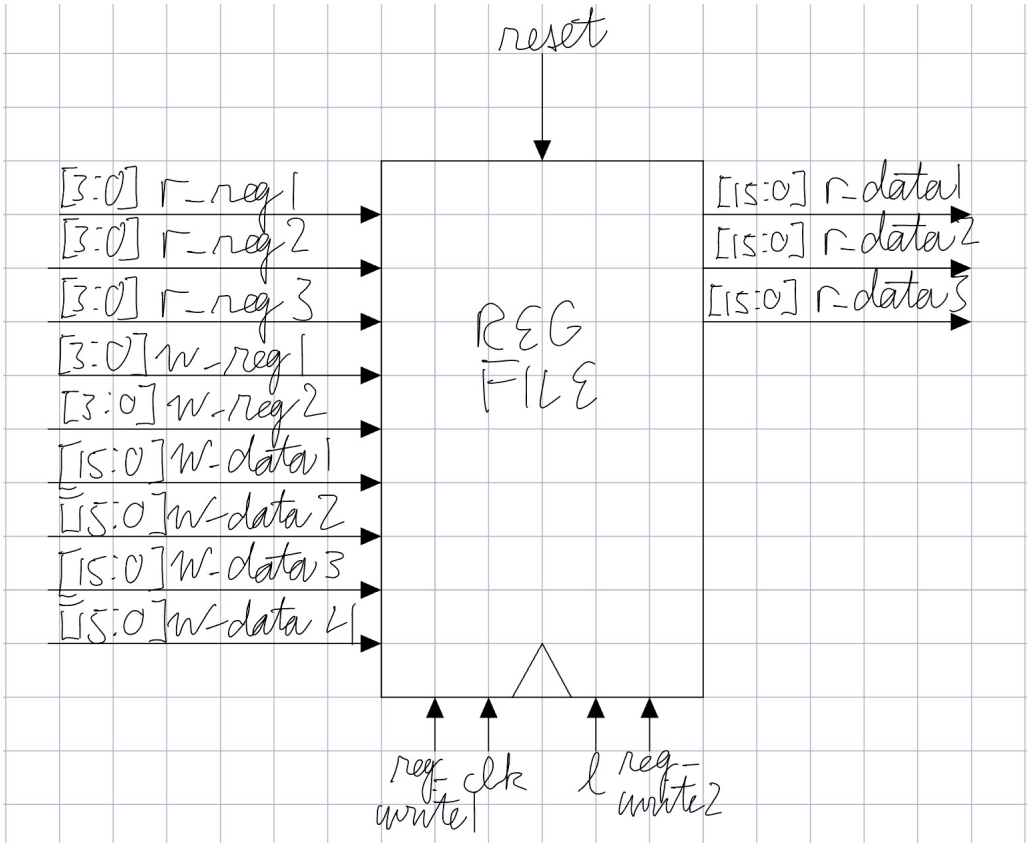


Figure 20: Sketch of the register file.

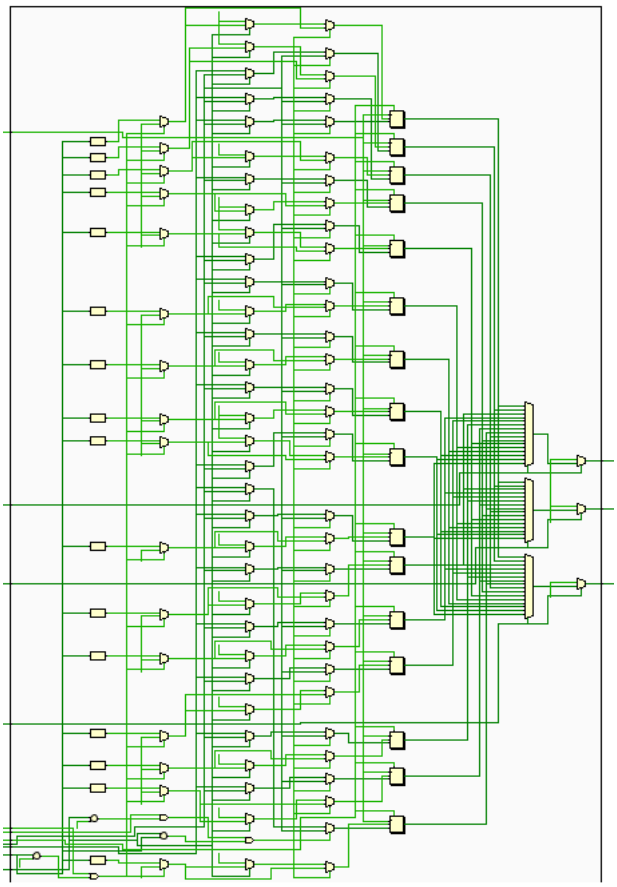


Figure 21: RTL model of the register file.

4.4.2 Verification

Test Plan

Register File Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	Reset	Reset should set all registers in the register file to 0
1.2	Read/Write Functionality	When the write signal is asserted, the register file should store values, regardless of the instruction given. When data is requested, it should output the data given the register
1.2.1	Separate Cycle Write/Read (Same Register)	Be able to write a value to a register in one cycle and then read it in the next
1.2.2	Separate Cycle Write/Read (Different Registers)	Be able to write a value to a register in one cycle and then read it in the next
1.2.3	r0 Constant	r0 should remain constantly 0 even when written to
1.2.4	Overwriting	Registers should be able to be overwritten (with the exception of r0)
1.2.5	Multi-Operand Function	Multiple registers should be able to be read and overwritten in the same clock cycle (MUL) or in different clock cycles
1.2.6	Unary Operations	When given a unary operation instruction (absx, notx), the 3rd operand position should output 0

Tests

A scoreboard was created to keep track of the expected values of the register file whenever it was written to or read from. It is also used to directly check with register file to see if what is read from the register file is what is expected. The tests were conducted to satisfy the test plan requirements like so:

- **1.1**- The reset signal was asserted and deasserted in 2 separate clock cycles. Iterating through the first 4 registers, it was checked that their values were 0.
- **1.2.1**- First, the two reg_write signals were set to high. Then, data was written to the register file. After one clock cycle, data was read from the same register. This was done for both read and write ports.
- **1.2.2**- First, the two reg_write signals were set to high. Then, data was written to the register file. After one clock cycle, data was read from a different register. This was done for both read and write ports.
- **1.2.3**- Both reg_write signals were set to high. Then, w_reg1 was set to 0 to write to register r0 through the first write port. Cycling through different write values, register r0 was checked. This was also done through the second write port.
- **1.2.4**- Both reg_write signals were set to high. The register file was preloaded with values. Each register in the register file was then overwritten with different values through the first write port. After that, this was repeated through the second write port.
- **1.2.5**- Both reg_write signals were asserted. Iterating through the first 8 registers, each write port was assigned some register number. When the register ports held the same value, it was tested that the first write port was prioritized to write to the register file. Otherwise, the registers specified by both write ports were written with the data corresponding to each write port.
- **1.2.6**- Both reg_write signals were set to high. The register specified by r_reg1 iterates over the register file while r_reg2 is set to 0.

4.5 Immediate Decoder

4.5.1 Design

The ISA uses a modified version of how ARMv7 encodes immediates, which involves rotating an 8-bit immediate in a 16-bit bus to get the desired immediate. Unlike ARMv7, which encodes immediates as an 8-bit value rotated right by an even number of bits (0, 2, 4, ...), this ISA allows an immediate to be shifted by any number from 0 to 15 bits. This gives full flexibility for forming 16-bit constants without the 2-bit rotation step limitation. From the instruction and before being sent to the ALU, immediates must be decoded from their 12-bit encoded value to their 16-bit true value.

Design Specification

- Purpose and Scope
  - The module is needed to convert an immediate encoded by an 8-bit immediate with a rotation to the intended 16-bit immediate that the ALU can manipulate.
  - Immediates are in the range of 0 to 32767
  - It is assumed that the immediate inputted is a valid immediate that can be encoded by the assembler
- Functional Requirements
  - The immediate decoder must take in an immediate and a rotation value, and output the immediate that it decodes to.
- Interface Specification
  - Inputs
    - imm8: 8-bit encoded immediate stored in instruction
    - rot: 4-bit value that tells how much to rotate imm8 by to get the decoded immediate
  - Outputs
    - imm: 16-bit decoded immediate

Implementation

The implementation is a simple rotator on a 16-bit register.

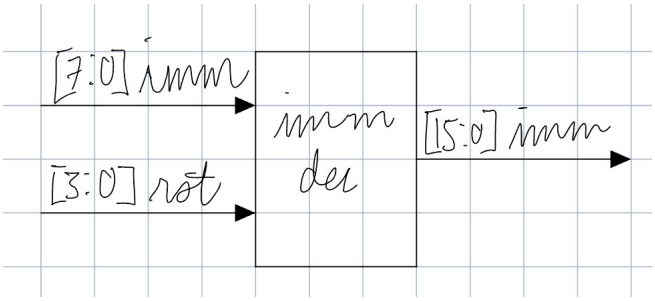


Figure 22: Sketch of the immediate decoder.

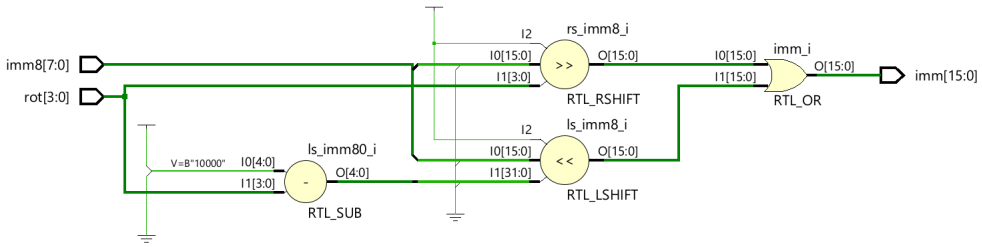


Figure 23: RTL model of the immediate decoder.

4.5.2 Verification

Test Plan

Immediate Decoder Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	Accurate Immediate Decoding	Valid immediates should be able to be accurately decoded for the entire range specified

Tests

- **1.1**- All possible immediates with their corresponding rot and imm8 values were generated in Python to serve as test vectors. The rot and imm8 values were inputted into the module, and the output was compared with its expected input.

4.6 Shifter

A shifter is hardware module that takes in data, shifts the data in a certain way, and then outputs the shifted data. If R_m is not an immediate, then the shifter provides more ways to manipulate data before it is sent to the ALU.

4.6.1 Design

Design Specification

- Purpose and Scope
  - Shifting allows for multiple instructions to be encoded in one, and is good for bit manipulation.
  - Having a separate shifter module simplifies the ALU, displacing the need for a 3rd register port and internal logic to support it.
- Functional Requirement
  - Given an input value, the shifter must either pass the value if the value originated from an immediate instruction or shift the value if the value and then pass the value if originated from a register
  - The amount a register value is shifted by is determined by an inputted shift type and the value recorded in R_s (a register that stores the amount to shift the input value) or an immediate (shift amount)
- Interface Specification
  - Inputs
    - R_m or imm: The value to be shifted by the shifter
    - shtype: The way that the shifter shifts R_m/imm (ror, asr, lsr, lsl)
    - r_shift: Determines whether or not the value comes from an immediate or a register
    - R_s or shamt: The amount to shift the R_m/imm by
  - Outputs
    - shifted_rm: R_m/imm shifted by the shifted

Implementation

The shifter decides where its data sources come from. Given that the third operand of the instruction is an immediate, it will just pass the immediate along

without shifting it. If the third operand is a register, it will shift the register value in some way specified by the shtype input. The amount specified can either come from a register or an immediate, which is chosen by the r_shift bit of the instruction.

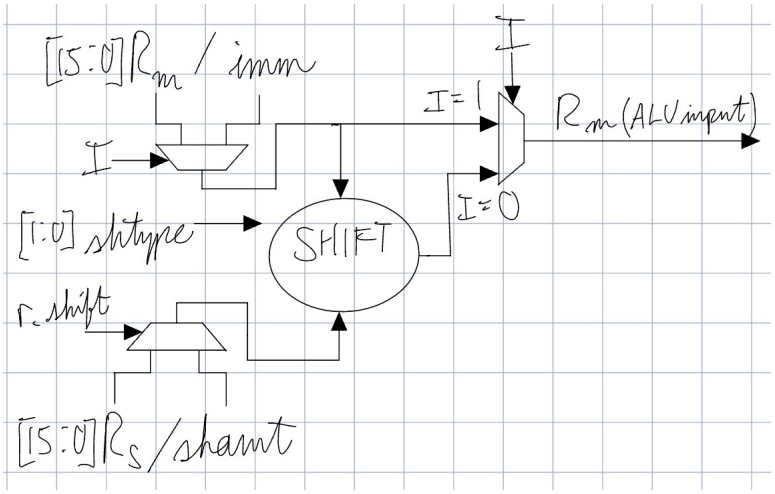


Figure 24: Sketch of shifter.

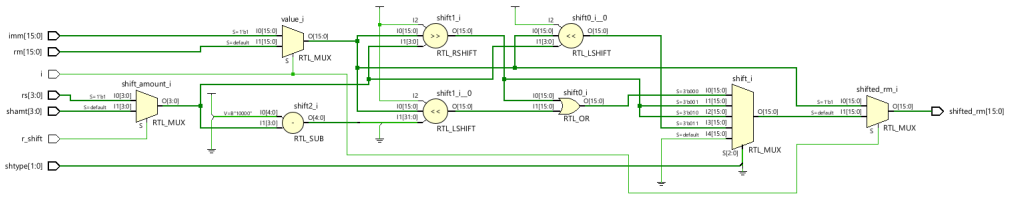


Figure 25: RTL model of the shifter.

4.6.2 Verification

Test Plan

Shifter Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	ror	- ror shifts should shift input data that is expected for rotate right
1.1.1	Immediate Input	- given an immediate input, the immediate should just be passed on to the ALU
1.1.2	Register Input	- Given a register input, the value stored in the register should be rotated right according to a given value
1.1.2.1	Shift Register	- Given a register input, the value stored in the register should be rotated right according to a value given from a shift register
1.1.2.2	Shift Amount	- Given a register input, the value stored in the register should be rotated right according to a value given by an immediate
1.2	asr	- asr shifts should shift input data that is expected for arithmetic shift right
1.2.1	Immediate Input	- given an immediate input, the immediate should just be passed on to the ALU
1.2.2	Register Input	- Given a register input, the value stored in the register should be arithmetic shifted right according to a given value
1.2.2.1	Shift Register	- Given a register input, the value stored in the register should be arithmetic shifted right according to a value given from a shift register
1.2.2.2	Shift Amount	- Given a register input, the value stored in the register should be arithmetic shifted right according to a value given by an immediate
1.3	lsr	- lsr shifts should shift input data that is expected for logical shift right
1.3.1	Immediate Input	- given an immediate input, the immediate should just be passed on to the ALU
1.3.2	Register Input	- Given a register input, the value stored in the register should be logical shifted right according to a given value
1.3.2.1	Shift Register	- Given a register input, the value stored in the register should be logical shifted right according to a value given from a shift register
1.3.2.2	Shift Amount	- Given a register input, the value stored in the register should be logical shifted right according to a value given by an immediate
1.4	lsl	- lsl shifts should shift input data that is expected for logical shift left
1.4.1	Immediate Input	- given an immediate input, the immediate should just be passed on to the ALU
1.4.2	Register Input	- Given a register input, the value stored in the register should be logical shifted left according to a given value
1.4.2.1	Shift Register	- Given a register input, the value stored in the register should be logical shifted left according to a value given from a shift register
1.4.2.2	Shift Amount	- Given a register input, the value stored in the register should be logical shifted left according to a value given by an immediate

Tests

Distinct sets of 16 values were used for R_m, immediates, R_s, and shamt. A scoreboard class was created to facilitate testing, having functions that verified functionality for immediate data input, register shift input, and immediate shift input. Taking in the shift type as input, the following 3 tests were applied for ROR, ASR, LSR, and LSL shifts.

- **1.X.1**- i is set to 1 (as expected for an immediate), and r_shift is given alternating values to demonstrate that the values coming from shifting have no affect in immediate mode. Iterating from 0 to 15, new data was set into r_m,

imm, r_s, and shamt. The output of the shifter was then compared with the expected value given by the scoreboard.

- **1.X.2.1**- i is set to 0 (as expected for a register), and r_shift is set to 1 (for a shift amount specified by a register). Iterating from 0 to 15, new data was set into r_m, imm, r_s, and shamt. The output of the shifter was then compared with the expected value given by the scoreboard.
- **1.X.2.2**- i is set to 0 (as expected for a register), and r_shift is set to 0 (for a shift amount specified by an immediate). Iterating from 0 to 15, new data was set into r_m, imm, r_s, and shamt. The output of the shifter was then compared with the expected value given by the scoreboard.

4.7 ALU

4.7.1 Design

The ALU is in charge of performing various operations specified by the instruction. Given values from either registers or immediates, the ALU must output the appropriate value. Details on each instruction can be found in the ISA Design section, and the expected ALU inputs/outputs are discussed in the design specification.

Design Specification

- Purpose and Scope
  - A module is needed to process data from the registers, and output results to either data memory or back to the registers.
- Functional Requirements
  - The ALU should be able to perform all the operations necessary for all RX and D instructions in the ISA (see below for specifics on each instruction)
  - Branching instructions will be put into a separate unit.
- Interface Specification
  - Inputs
    - r_data1: The first 16-bit data to be inputted into the ALU
    - r_data2: The second 16-bit data to be inputted into the ALU
    - r_data3: The third 16-bit data to be inputted into the ALU
    - s: Determines whether or not to set NZCV flags
    - Cin: Determines whether or not to put in a carry-in from the previous instruction.
    - opcode: Determines the operation to perform on the operands
  - Outputs
    - w_data1: The first 16-bit data to be written back to the registers
    - w_data2: The second 16-bit data to be written back to the registers
    - NZCV: NZCV flags after operation is performed

ALU Function Specification

- RX-Type
  - addx
    - It should add 2 integer values specified by r_data1 and r_data2 and output the sum in w_data1 with appropriate NZCV flags.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
  - subx
    - It should subtract 2 integer values specified by r_data1 and r_data2 and output the difference in w_data1 with appropriate NZCV flags.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
  - mulx
    - It should multiply 2 integer values specified by r_data1 and r_data2, and store the product's upper 16 bits in a register specified by w_data1 and the product's lower 16 bits specified by w_data2.
    - The operation should never affect NZCV flags.
  - divx
    - It should divide an integer value specified by r_data1 by an integer specified by r_data2, and output the quotient to w_data1 and the remainder to w_data2.
    - The operation should never affect NZCV flags.
  - absx
    - It should take the absolute value of an integer specified by r_data1, and output the absolute value to w_data1.
    - The operation should never affect NZCV flags.
  - adcx
    - It should add 2 integer values specified by r_data1 and r_data2, and a carry-in value from a previous operation, and then output the sum in w_data1 with appropriate NZCV flags.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
  - sbcx

    - It should subtract 2 integer values specified by r_data1 and r_data2, and a carry-in value from a previous operation, and then output the difference in w_data1 with appropriate NZCV flags.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
  - cmpx
    - It should compare 2 integer values specified by r_data1 and r_data2 and set NZCV flags accordingly.
  - notx
    - It should take the bitwise not of an integer specified by r_data1, and output the absolute value to w_data1.
    - The operation should never affect NZCV flags.
  - andx
    - It should take the bitwise and of two values specified by r_data1 and r_data2, and output the result to w_data1.
  - orrx
    - It should take the bitwise or of two values specified by r_data1 and r_data2, and output the result to w_data1.
  - xorx
    - It should take the bitwise xor of two values specified by r_data1 and r_data2, and output the result to w_data1.
- D-Type
  - ldw
    - It should calculate the address in memory using the base address specified by r_data1, and adding an offset specified by r_data2.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
  - ldb2l
    - It should calculate the address in memory using the base address specified by r_data1, and adding an offset specified by r_data2.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
  - ldb2h

    - It should calculate the address in data memory using the base address specified by r_data1, and adding an offset specified by r_data2.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
- stw
    - It should calculate the address in data memory using the base address specified by r_data1, and adding an offset specified by r_data2.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
- stb2l
    - It should calculate the address in data memory using the base address specified by r_data1, and adding an offset specified by r_data2.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.
- stb2h
    - It should calculate the address in data memory using the base address specified by r_data1, and adding an offset specified by r_data2.
    - For the case of shifting r_data2, r_data3 will determine the shift amount.

Implementation

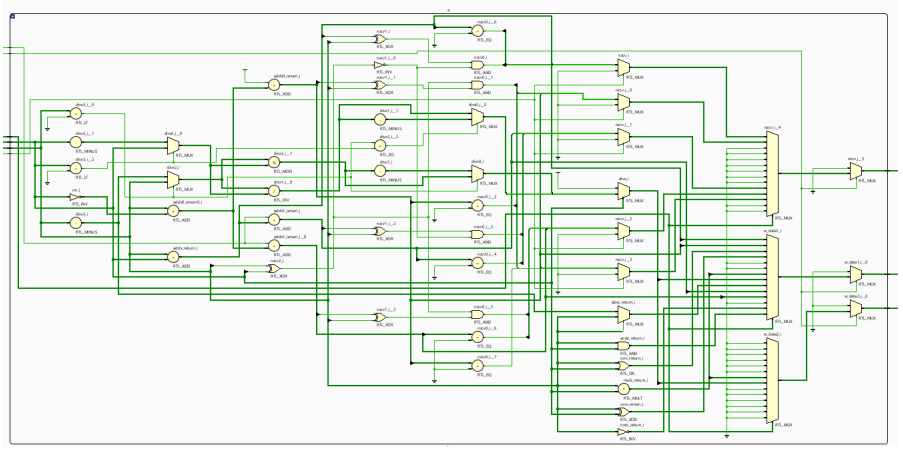


Figure 26: RTL model of ALU.

4.7.2 Verification

Test Plan

ALU Test Plan: RX Instructions		
#	**Title**	**Description**
1	Core Features: RX Functionality	
1.1	addx/subx	- Add two integers together and produce appropriate NZCV flags according to what is expected from 2's complement addition
1.1.1	Sum Output	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.1.2	C Flag	-Adding two integers should produce a carry out value that is expected for all inputs, and use that to set the C flag
1.1.3	V Flag	- Adding two integers should produce a carry out value that is expected for all inputs, and use that to set the V flag
1.1.3.1	V Flag: Adding Two Positives	- Adding two positive integers should produce a set V flag if the resulting sum is negative
1.1.3.2	V Flag: Adding Two Negatives	- Adding two negative integers should produce a set V flag if the resulting sum is positive
1.1.3.3	V Flag: Mixed Signs	- Adding two integers whose signs differ should not set the V flag
1.1.4	Z Flag	- If the sum of two integers is 0, it should set the Z flag
1.1.5	N Flag	- If the sum of two integers is negative, it should set the N flag
1.2	mulx/divx	- Multiplying two integers together should produce a product that is expected from the multiplication, where the upper 16 bits of the product are stored in w_data1, and the lower 16 bits of the product are stored in w_data2
1.2.1	Multiply/divide 2 positive numbers	
1.2.2	Multiply/divide 2 negative numbers	
1.2.3	Multiply/divide positive by negative number	
1.2.4	Multiply/divide negative by positive number	
1.3	absx	- Given any integer, it should take the absolute value of it, and store it into w_data1.
1.4	adcx/sbcx	- Add two integers together with carry flag from a previous instruction and produce appropriate NZCV flags according to what is expected from 2's complement addition.
1.4.1	Sum Output	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.4.2	C Flag	-Adding two integers should produce a carry out value that is expected for all inputs, and use that to set the C flag
1.4.3	V Flag	- Adding two integers should produce a carry out value that is expected for all inputs, and use that to set the V flag
1.4.3.1	V Flag: Adding Two Positives	- Adding two positive integers should produce a set V flag if the resulting sum is negative
1.4.3.2	V Flag: Adding Two Negatives	- Adding two negative integers should produce a set V flag if the resulting sum is positive
1.4.3.3	V Flag: Mixed Signs	- Adding two integers whose signs differ should not set the V flag
1.4.4	Z Flag	- If the sum of two integers is 0, it should set the Z flag
1.4.5	N Flag	- If the sum of two integers is negative, it should set the N flag
1.5	cmpx	- Comparing any two integers (subtract without storing result), should set the appropriate NZCV flags
1.6	notx	- Taking the bitwise not of any integer should produce the expected output
1.7	andx	- Taking the bitwise and of any two sets of 16-bit data should produce the expected output
	orrx	- Taking the bitwise or of any two sets of 16-bit data should produce the expected output
	xorx	- Taking the bitwise xor of any two sets of 16-bit data should produce the expected output

ALU Test Plan: D Instructions		
#	**Title**	**Description**
1	Core Features: D Functionality	
1.1	ldw	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.2	ldb2h	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.3	ldb2l	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.4	stw	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.5	stb2h	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1
1.6	stb2l	- Adding two integers should output the expected sum as a [15:0] value, and store the sum in w_data1

Tests

A scoreboard was used to check the functionality of every opcode input into the ALU.

- **1.1.1-1.1.5**- For each of these tests, Cin was set to 0, and s was set to 1. When conducting an addx test, the opcode was set to ADDX, and when conducting a subx test, the opcode was set to SUBX. 12 datasets of 16 integers were used to thoroughly verify the sums, differences, and NZCV flags of addx and subx operations. The datasets had characteristics that include:
  - Addition and subtraction where operands have every combination of signs.
  - Addition and subtraction result in both signed and unsigned overflow in both negative and positive directions.
  - Addition and subtraction result in 0.
- **1.2.1-1.2.4**- For each of these tests, Cin was set to 0, and s was set to 0. When conducting an mulx test, the opcode was set to MULX, and when conducting a divx test, the opcode was set to DIVX. 2 datasets of 20 values were used. For the first dataset, the first 16 values are 0-15, and the last 4 are very large values for a 16-bit signed integer. The second dataset is the negative version of the first dataset. Every combination of inputs was verified for multiplication and division.
- **1.3**- For these tests, Cin was set to 0, and s was set to 0. The opcode was set to ABSX. A dataset from -8 to 7 was used to verify the output of abx.

- **1.4.1-1.4.5**- For each of these tests, Cin was set to 1, and s was set to 1. When conducting an addx test, the opcode was set to ADDX, and when conducting a subx test, the opcode was set to SUBX. 12 datasets of 16 integers were used to thoroughly verify the sums, differences, and NZCV flags of addx and subx operations. The datasets and conditions used were the same as the ones for 1.1.1-1.1.5.

- **1.5**- For these tests, Cin was set to 0, and s was set to 0. The opcode was set to CMPX. Two datasets were used to verify cmpx. One dataset had values from -8 to 7, and the other had values from 8 to -7.

- **1.6**- For these tests, Cin was set to 0, and s was set to 0. The opcode was set to NOTX. The dataset used to verify notx was the same one as absx.

- **1.7**- For these tests, Cin was set to 0, and s was set to 0. The opcode was set to notx. The opcode was set to ANDX, ORRX, or XORX depending on the test being conducted. Two datasets of small positive and negative numbers were used to verify the 3 operations.

4.8 op2 Decoder

4.8.1 Design

Design Specification

- Purpose and Scope
  - It is an organizational unit that connects immediate decoders, shifter, and ALU.
  - It is needed to make the design more modular, and to decrease the length of the execution stage, when this processor is pipelined.
- Functional Requirements
  - The decoder should connect several of the aforementioned in the previous bullet point together.
  - The decoder should take in bits from the instruction that detail immediate encoding, register encoding, etc. to produce the final operand value that the ALU will operate on.
- Interface Specification
  - Inputs
    - [7:0] imm_m: Second operand input into the ALU if it is an immediate
    - [3:0] rot_m: Rotation to apply to the second operand input if it was an immediate
    - [15:0] rm: Second operand input into the ALU
    - i: Bit determining if the second operand input is an immediate
    - [1:0] shtype: Bits determining the shift type (ROR, ASR, LSR, LSL)
    - r_shift: Bit determining if the rotation applied to the second operand originates from a register or an immediate
    - [7:0] imm_s: Shift applied to the register
    - [3:0] rot_s: Rotation applied to the shift applied to the register
  - Outputs
    - rm_dec: Decoded version of the second operand

Implementation

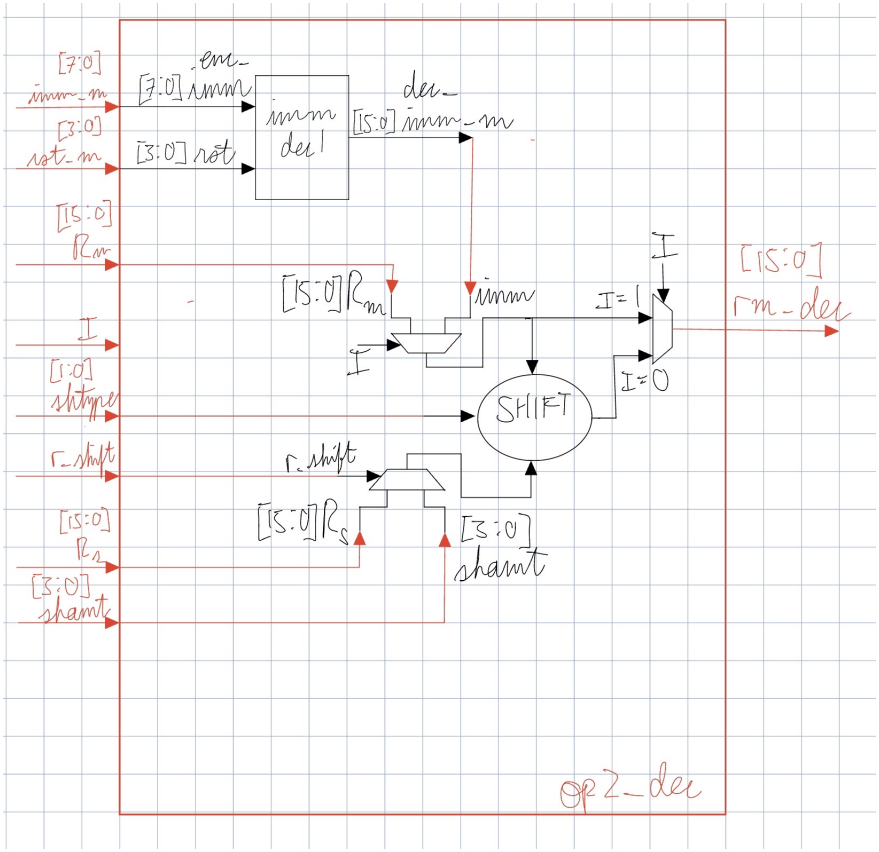


Figure 27: Sketch of op2 decoder.

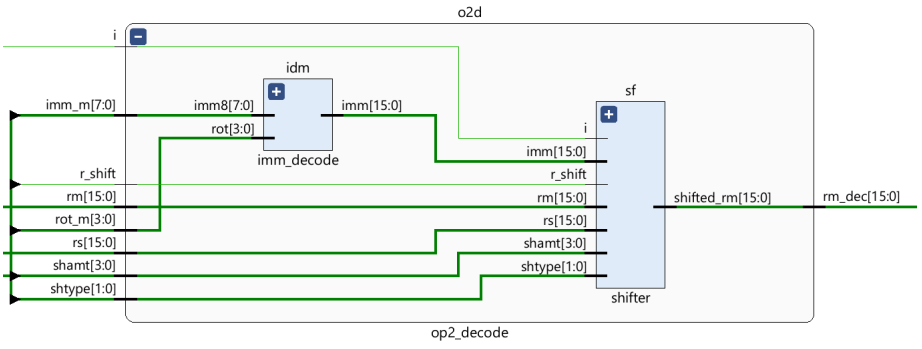


Figure 28: RTL model of op2 decoder.

4.8.2 Verification

Test Plan

op2 Decoder Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	Rm Immediate Function	When given an instruction like addx-al r1, r0, #4, the op2 decoder should decode immediates as expected of the given immediate encoding, and then output it without further modification
1.2	Rm Register Function	
1.2.1	Rm Register - No modification	When given an instruction like addx-al r1, r0, r3 the op2 decoder should output the original value contained in Rm without modification
1.2.2	Rm Register - Shift by Immediate	When given an instruction like addx-al r1, r0, r3, lsl #4 the op2 decoder should output a shifted version (shifted according to the lsl argument) of the original value contained in Rm
1.2.2.1	Shift: ROR	Test I/O for instructions like addx-al r1, r0, r3, ror #4
1.2.2.2	Shift: ASR	Test I/O for instructions like addx-al r1, r0, r3, asr #5
1.2.2.3	Shift: LSR	Test I/O for instructions like addx-al r1, r0, r3, lsr #6
1.2.2.4	Shift: LSL	Test I/O for instructions like addx-al r1, r0, r3, lsl #7
1.2.3	Rm Register - Shift by Rs	When given an instruction like addx-al r1, r0, r3, lsl r9 the op2 decoder should output a shifted version (shifted according to the lsl argument) of the original value contained in Rm
1.2.3.1	Shift: ROR	Test I/O for instructions like addx-al r1, r0, r3, ror r5
1.2.3.2	Shift: ASR	Test I/O for instructions like addx-al r1, r0, r3, asr r6
1.2.3.3	Shift: LSR	Test I/O for instructions like addx-al r1, r0, r3, lsr r7
1.2.3.4	Shift: LSL	Test I/O for instructions like addx-al r1, r0, r3, lsl r8

Tests

A scoreboard was used to check the functionality of the entire unit. It comes with built in functions to shift operands, and has check functions that mimic the cases for when rm is shifted by immediate and by register.

For each test, a small program was written using the ISA design, and was assembled into machine code. An example test program is shown below. The machine code from the test program was put into the testbench where the machine code was sectioned into its fields using a struct to organize the instruction fields. The op2 of each instruction was put into another struct, the structure of which depended on the test that was being performed (shift by register or by immediate). Using these new organized structs in memory, the fields of the instructions were passed into the DUT, along with directed test vectors for rm, rs, and shamt.

```
  addx-al r1, r2, r3, lsl r1

addx-al r1, r2, r3, lsl r1

addx-al r1, r2, r3, lsl r2

addx-al r1, r2, r3, lsl r3

addx-al r1, r2, r3, lsl r4
```

```
addx-al r1, r2, r3, lsl r5

addx-al r1, r2, r3, lsl r6

addx-al r1, r2, r3, lsl r7

addx-al r1, r2, r3, lsl r8

addx-al r1, r2, r3, lsl r9

addx-al r1, r2, r3, lsl r10

addx-al r1, r2, r3, lsl r11

addx-al r1, r2, r3, lsl r12

addx-al r1, r2, r3, lsl r13

addx-al r1, r2, r3, lsl r14

addx-al r1, r2, r3, lsl r15
```

4.9 ALU Top

4.9.1 Design

To separate ALU function from choosing the appropriate opcode, a top module was created to simplify the interface between the ALU with the rest of the processor. This section only covers the simplest version of ALU Top, and more features will be added in the pipelining section.

Design Specification

- Purpose and Scope
  - The module is needed to simplify the interface between the ALU and the rest of the processor.
  - It should implement something to identify what kind of instruction is being inputted into the ALU, and choose the opcode accordingly.
- Functional Requirements
  - The module should take in inputs originating from the instruction and output processed data from the ALU.
- Interface Specification
  - Inputs
    - [15:0] rn: First operand input into the ALU
    - [15:0] dec_rm: Second operand input into the ALU (after being decoded by op2 decoder)
    - s: Bit determining if the nzcv flags are set after the operation
    - Cin: Carry-in bit from a previous operation
    - en: Enables the ALU
    - [1:0] instr_class: Bits determining if the originating instruction is RX, RF, D, or B
    - [3:0] opcode: Bits detailing what operation to perform in the ALU
  - Outputs
    - [15:0] w_data1: First output data from ALU operation
    - [15:0] w_data2: Second output data from ALU operation
    - [3:0] NZCV: Negative, Zero, Carry, and Overflow flags set after an operation

Implementation

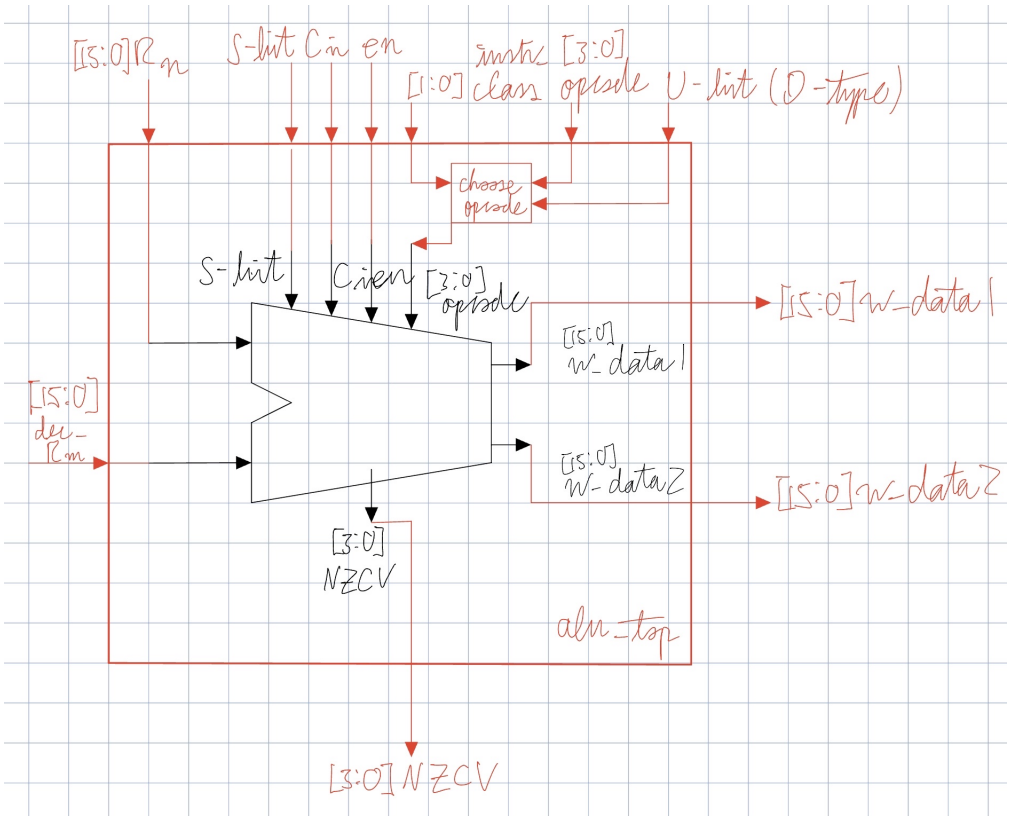


Figure 29: Sketch of ALU top module.

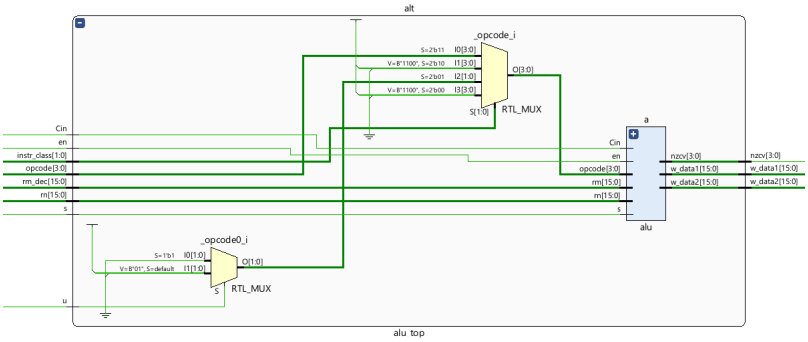


Figure 30: RTL model of ALU top module.

4.9.2 Verification

Test Plan

ALU Top Test Plan		
#	**Title**	**Description**
1	Core Features	
1.1	RX	When class type is set to RX, the opcode is passed through
1.2	D	When class type is set to D, the opcode is set to either ADD or SUB depending on the u bit
1.2.1	Positive Address Offset	When u = 1, the address should be incremented by the offset to get the final address
1.2.2	Negative Address Offset	When u = 0, the address should be decremented by the offset to get the final address
1.3	B	Opcode should be set to NOOP

Tests

A scoreboard was used to check the functionality of the entire unit. It comes with built in functions to verify that the appropriate opcodes are passed into the ALU according to each instruction.

For each test, a program was written using the ISA design, and was assembled into machine code. The program contains most instructions for RX, D, and B type instructions. The test program is shown below.

```
regx:

addx-al r1, r2, r3

addx-al r10, r14, #9

addx-al r1, r2, r3, lsl r4

addx-al r1, r2, r3, lsl #4

addx-al r1, r5, #452



addx.s-al r1, r2, r3

addx.s-al r10, r14, #9

addx.s-al r1, r2, r3, lsl r4
```

```
addx.s-al r1, r2, r3, lsl #4
addx.s-al r1, r5, #452


subx-al r1, r2, r3
subx-al r10, r14, #9
subx-al r1, r2, r3, lsl r4
subx-al r1, r2, r3, lsl #4
subx-al r1, r5, #452

subx.s-al r1, r2, r3
subx.s-al r10, r14, #9
subx.s-al r1, r2, r3, lsl r4
subx.s-al r1, r2, r3, lsl #4
subx.s-al r1, r5, #452


adcx-al r1, r2, r3
adcx-al r10, r14, #9
adcx-al r1, r2, r3, lsl r4
adcx-al r1, r2, r3, lsl #4
adcx-al r1, r5, #452
```

```
adcx.s-al r1, r2, r3

adcx.s-al r10, r14, #9

adcx.s-al r1, r2, r3, lsl r4

adcx.s-al r1, r2, r3, lsl #4

adcx.s-al r1, r5, #452


sbcx-al r1, r2, r3

sbcx-al r10, r14, #9

sbcx-al r1, r2, r3, lsl r4

sbcx-al r1, r2, r3, lsl #4

sbcx-al r1, r5, #452


sbcx.s-al r1, r2, r3

sbcx.s-al r10, r14, #9

sbcx.s-al r1, r2, r3, lsl r4

sbcx.s-al r1, r2, r3, lsl #4

sbcx.s-al r1, r5, #452


mulx-eq r1, r2, r3

divx-eq r1, r2, r3


absx-al r9, r11
```

```
cmpx-al r12, r9

cmpx-al r12, #99


notx-eq r2, r1

andx-al r3, r1, r2

andx-al r2, r1, #0x00FF

orrx-al r3, r1, r2

orrx-al r2, r1, #0x1100

xorx-al r3, r1, r2

xorx-al r2, r1, #0x0FF0



regd:

ldw-al r1, [r14, #2]

ldw-al r1, [r14, r2]

ldw-al r1, [r14, r2, lsl #8]

ldw-al r1, [r14, r2, lsl r9]

ldw-al r1, [r14, #-2]

ldw-al r1, [r14, -r2]


ldb2l-al r1, [r14, #2]
```

```
ldb2l-al r1, [r14, r2]

ldb2l-al r1, [r14, r2, lsl #8]

ldb2l-al r1, [r14, r2, lsl r9]

ldb2l-al r1, [r14, #-2]

ldb2l-al r1, [r14, -r2]


ldb2h-al r1, [r14, #2]

ldb2h-al r1, [r14, r2]

ldb2h-al r1, [r14, r2, lsl #8]

ldb2h-al r1, [r14, r2, lsl r9]

ldb2h-al r1, [r14, #-2]

ldb2h-al r1, [r14, -r2]

regb:

bx-al lr

b-eq other

b-al regd

subx.s-al r0, r1, r2

b-eq regb

bl-al regx

other:

xorx-al r3, r1, r2
```

- **1.1**- Every RX instruction opcode was successfully passed into the ALU.
- **1.2**- Every D instruction passed ADDX for load instructions that have an address offset that is positive, and SUBX for load instructions that have an address offset that is negative.
- **1.3**- NOOP was successfully passed to the ALU for each branching instruction.

4.10 Main Control Unit

The main control unit is in charge of generating and routing control signals needed throughout the processor. This includes signals to allow the register to be written to, signals to read and write from memory, and more. Note: This section is unfinished, and needs to be updated once other units are implemented.

4.10.1 Design

Design Specification

- Purpose and Scope
  - A central module that focuses on the generation and routing of control signals is needed to keep the top-level organization easy to read and debug.
  - The module focuses only on control signal manipulation, and should not modify any outside signals, like instructions.
- Functional Specifications
  - The module should route and generate the following signals:
    - Signals pertaining to writing to either write port of the register file
    - Signals pertaining to writing and reading from the data memory
    - Signals controlling which data is written to the register file
    - Signals choosing what value the program counter updates with (branching case)
    - Possibly more!
- Interface Specifications
  - Inputs
    - [31:0] instruction: Input instruction
    - [3:0] nzcv: Previous state of NZCV conditions flags
  - Outputs
    - reg_write1: Bit to control to write to the register through its first write port
    - reg_write2: Bit to control to write to the register through its second write port
    - mem_write: Bit to control writing to the data memory

* mem2reg: Bit to control where the data being written to the register is originating from (0 for ALU, 1 for data memory)
* i: Tells op2dec whether or not the second operand is a register or an immediate
* s_or_u: Holds the place of two flags. For RX instructions, it tells the ALU to allow the operation to set NZCV flags. For D instructions, it tells the ALU whether to add or subtract the address offset
* [1:0] instr_class: Two bits defining which of RX, RF, D, and B the instruction is
* [3:0] opcode: Determines the operation to perform on the operands
* alu_en: Determines whether or not the ALU performs its operation
* mem_read: Determines whether or not the data memory is able to be read for the instruction
* [1:0] byte˙sel: Determines which bytes are selected to be written/read to/from for a register
* cond˙met: Determines if the condition specified by the NZCV flags and the instruction cond flag is met
* branch: Determines if the processor needs to branch to another instruction based on the instruction class of the instruction
* stall: Determines if the processor needs to stall an instruction

Implementation

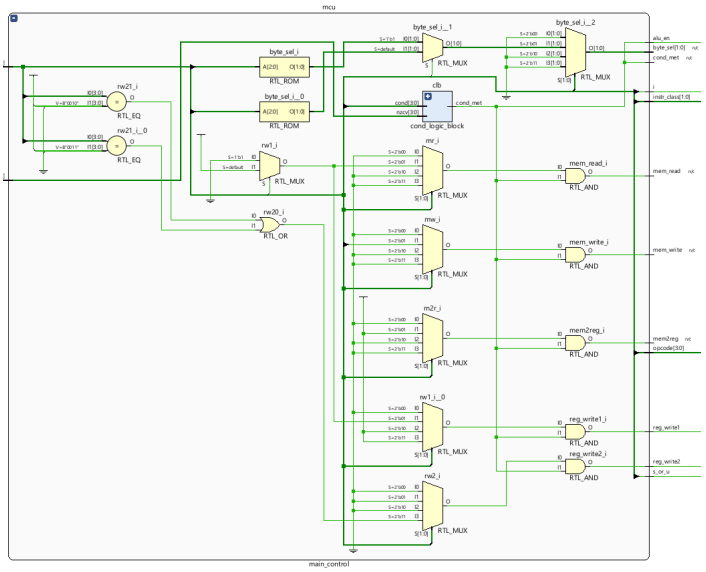


Figure 31: RTL of main control unit.

4.10.2 Verification

Test Plan

Main Control Unit Test Plan		
#	**Title**	**Description**
1	reg_write1, reg_write2, mem_write, mem2reg	- The proper control signals should be outputted for any instruction inputted into the module. When talking about control signal vectors, the format is {reg_write1, reg_write2, mem_write, mem2reg}
1.1	RX instruction	- All RX instructions should output 4'b1000, with the exception of DIV and MUL instructions, which should both be 4'b1100
1.2	RF instruction	TBD
1.3	D instruction	- LDR instructions should output 4'b1001, and STR instructions should output 4'b0011
1.4	B instruction	- BX instructions should output 4'b0000

Tests

- **1.1-1.4**- A simple test program was written using the ISA design, and was assembled into machine code. It contains all instructions that have unique changes to the control output. ADDX was repeated with different argument inputs just to show that the inputs do not affect the MCU output. All main control unit outputs were as expected. The test program is shown below:

```
    addx-al r1, r0, #200
addx-al r2, r0, #70
addx-al r3, r1, r2
mulx-al r3, r4, r5
divx-al r6, r7, r8
ldw-al r9, [r10, #8]
stw-al r1, [r4, #16]
ldb2l-al r2, [r5, #12]
stb2l-al r9, [r10, #16]
ldb2h-al r2, [r5, #12]
stb2h-al r9, [r10, #16]
bx-al lr
```

4.11 Condition Logic Block

4.11.1 Design

The ISA has a built-in way to skip instructions if the instruction's cond bits don't match the current state of the NZCV flags.

Design Specification

- Purpose and Scope
  - A submodule for the main control unit is needed to organize all of the combinational logic for the conditional execution feature in each instruction.
- Function Specification
  - The submodule should take the cond bits from an instruction and output a flag that represents whether or not the instruction executes.
- Interface Specification
  - Inputs
    - [3:0] cond: cond flag originating from the instruction
    - [3:0] nzcv: Current state of NZCV flags
  - Outputs
    - cond_met: Flag dictating if the NZCV flags match the cond specified by the instruction

Implementation

4.11.2 Verification

Test Plan

Condition Logic Block Test Plan		
#	**Title**	**Description**
1	cond testing	- The output of the unit should be what is expected from comparing the cond bits to the state of the nzcv flags

Tests

- A scoreboard with a check function and a golden model of the conditional logic block was used to verify each possible combination of nzcv and cond values.

4.12 Data Memory

4.12.1 Design

Data memory is composed of 256 possible locations, each location holding 1 byte.

Design Specification

- Purpose and Scope
  - The ISA needs a data memory storage to load and store data from.
  - The unit will only store data memory. Instructions are found in the instruction memory unit.
- Function Specification
  - The unit should be able to be written to (in the case of store instructions).
  - The unit should be able to be read from (in the case of load instructions).
  - The unit should contain all the control logic necessary for different load-/store instructions.
    - For store instructions, when given the 16-bit input:
      - The unit should have 256 locations, each holding 1 byte of data, with 1 bit to represent the validity of the data.
      - If the instruction is stw, the 16-bit input should be stored at the specified address, where the most significant byte is stored in the address with the lowest value.
      - If the instruction is stb2h, the 16-bit input's most significant byte should be selected and stored at the specified address.
      - If the instruction is stb2l, the 16-bit input's least significant byte should be selected and stored at the specified address.
      - If the instruction is ldw, two 8-bit words should be selected from memory, where the address inputted into the data memory is the most significant byte, and the following address is the least significant byte. If there is no valid data located at either memory address used in the load, the value read is 16'hffff.
      - If the instruction is ldb2h, the unit outputs the data byte to the most significant bits of the data bus. The remaining bits are set to 0.

        - If the instruction is ldb2l, the unit outputs the data byte to the least significant bits of the data bus. The remaining bits are set to 0.
    *

- Interface Specification
    - Inputs
        * [15:0] w_data: 16-bit data to write into the data memory
        * [15:0] addr: 16-bit address to read/write from/to memory
        * mem_write: Control signal specifying whether or not to write to the memory
        * mem_read: Control signal specifying whether or not to read from memory
        * [1:0] byte˙sel: Control signal specifying which bytes to read/write from/to memory
        * clk: Clock signal
        * reset: Synchronous reset signal
    - Outputs
        * [15:0] r_data: 16-bit data to read from the data memory

Implementation

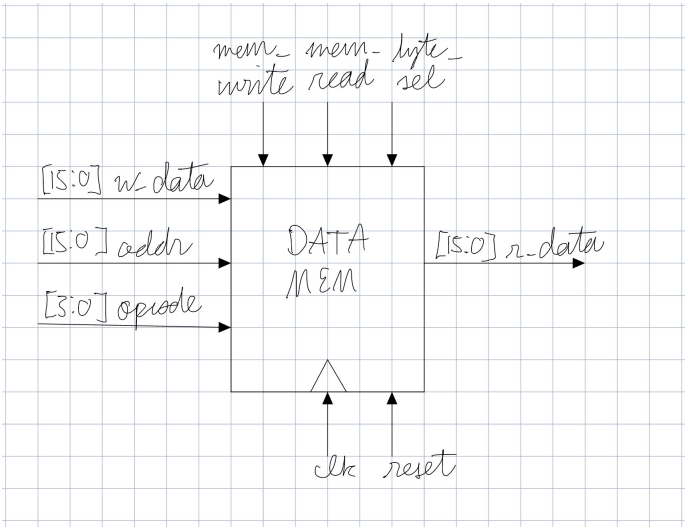


Figure 32: Sketch of data memory.

4.12.2 Verification

Test Plan

Data Memory Test Plan		
#	**Title**	**Description**
1	Writing	- Writing to the data memory should transfer the correct data from the register file to the data memory at the specified address location inputted
1.1	stw	- Writing to the data memory should transfer an entire word from the register file to the data memory, where the most significant byte of the register is stored at the specified address location inputted, and the least significant byte of the register is stored at the next address downstream.
1.2	stb2h	- Writing to the data memory should transfer an one byte from the register's most signifcant byte to the data memory at the specified address location inputted
1.3	stb2l	- Writing to the data memory should transfer an one byte from the register's least signifcant byte to the data memory at the specified address location inputted
2	Reading	- Reading from the data memory should transfer the correct data from the data memory to the register file at the specified address location inputted
2.1	ldw	- Reading the data memory should transfer an entire word from the data memory to the register file, where the most significant byte read to the register is stored at the specified address location inputted, and the least significant byte read to the register is stored at the next address downstream
2.2	ldb2h	- Reading to the data memory should transfer an one byte from the data memory at the specified address to the most significant byte of r_data, and the least significant byte should be filled with 0s
2.3	ldb2l	- Reading to the data memory should transfer an one byte from the data memory at the specified address to the least significant byte of r_data, and the most significant byte should be filled with 0s

Tests

- **Tests 1.1-1.3**- A task was written with an input for byte_sel to test each of the str instructions. Unique data was written to each memory location sequentially depending on the test. Then, a scoreboard with its own memory was loaded with the same data combinationally. The mems of the DUT and the scoreboard were compared to verify equality of both the datasets and the valid bits. With the exception of calling stw on address 255 (something not supported by the ISA), all tests passed.

- **Tests 2.1-2.3**- A task was written with an input for byte_sel to test each of the ldr instructions. Unique data was written to each memory location sequentially depending on the test. Then, a scoreboard with its own memory was loaded with the same data combinationally. The outputs of the read and the expected memory location in the scoreboard were compared to verify equality of both the read and the valid bits. With the exception of calling ldw on address 255 (something not supported by the ISA), all tests passed.

4.13 Branching Unit

The branching unit is in charge of going to different instructions in different locations of instruction memory, given that there are labels in the assembly script.

4.13.1 Design

Design Specification

- Purpose and Scope
  - The ALU takes in 16-bit operands, while the offset of a branching instruction is 24 bits wide. Because of this difference, a module separate from the ALU must be used to calculate the next address of the instruction if a branching instruction is called.
- Function Specification
  - Given a branching instruction offset, the branching unit should calculate the address of the next instruction (2's complement adder).
  - Depending on the instruction, the operands can be 16-bit or 22-bit.
    - If the R-bit is 1, the PC offset will be 16-bit (from the register file). If the R-bit is 0, then it will be a 10-bit offset added to PC
- Interface Specification
  - Inputs
    - [15:0] pc_next: Next program counter value that would be used if there were no branching
    - [15:0] pc: Program counter value
    - [9:0] instr_offset: Offset stored in the branching instruction
    - [15:0] rb: Register value containing the offset to be used in branching
    - R: Bit from the instruction that tells if the module to use an offset stored in a register
    - branch: Control flag that tells the branching unit to branch instead of using the next value of pc
  - Outputs
    - [15:0] out_pc: The next program counter value the processor should use
    - [15:0] lr: The link register value set if a branch with link is called

Implementation

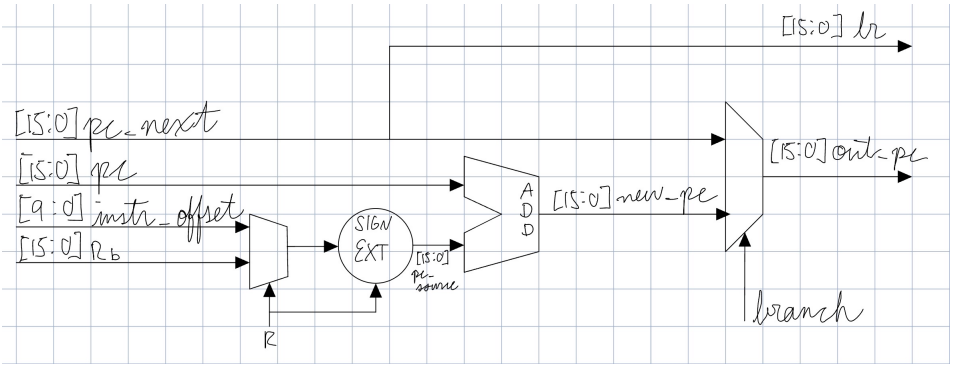


Figure 33: Sketch of the branching unit

4.13.2 Verification

Test Plan

Branching Unit Test Plan		
#	**Title**	**Description**
1	Case: No branching	- The output PC should just be the next value of program counter (pc + 4). - The output LR should just be pc + 4
2	Case: Register offset	- The output of the unit should be the program counter added to a twice-shifted value originating from a register. - The output LR should just be pc + 4
3	Case: Label offset	- The output of the unit should be the program counter added to a 1twice-shifted 6-bit value originating from a 10-bit offset stored in an instuction. - The output LR should just be pc + 4

Tests

For each test, a scoreboard with a checker was used to verify the outputs. The PC is set to 0 initially and is incremented by 4 for each iteration in each test. The new output PC is checked by using two vectors that contain multiples of 4 and 8 for instr_offset and R_b respectively. Each input is set to one of the values contained in these vectors.

- **Test 1**: Iterating 16 times, it is verified that the LR and PC generated from the DUT match expected LR and output PC, which are both calculated by adding 4 to PC.
- **Test 2**: Iterating 16 times, it is verified that the LR and PC generated from the DUT match expected LR and output PC, where expected PC is calculated by adding a twice right shifted register vector element to the expected PC and the expected LR is PC + 4.

- **Test 3**: Iterating 16 times, it is verified that the LR and PC generated from the DUT match expected LR and output PC, where the expected PC is calculated by adding a twice right shifted instruction offset vector element to the expected PC and the expected LR is PC + 4.